

Dokumentasi Lengkap — es-test: Secure 3-Node ElasticSearch Cluster on AWS

Klasifikasi: **INTERNAL**

Take-home test Infrastructure Engineer. Repo: `es-test/` (Terraform + Ansible), region `ap-southeast-3`.

Daftar Isi

1. Arsitektur & Alur Kerja Keseluruhan
2. Penjelasan Source Code — Stack per Stack
3. Cara Menggunakan & Integrasi
4. Monitoring & Alerting — Implementasi dan Flow
5. Cara Debugging Jika Ada Issue
6. Mekanisme Failover

1. Arsitektur & Alur Kerja Keseluruhan

Repo ini terdiri dari **4 stack independen**, masing-masing punya `terraform/` + `ansible/` + `deploy.sh` sendiri, dan harus di-apply **berurutan** karena stack turunan membaca state stack sebelumnya via `terraform_remote_state` (backend lokal, baca langsung file `.tfstate`):

- | | |
|-------------------------------------|---|
| 1. <code>es-test/</code> | -> cluster ElasticSearch 3-node (fondasi: VPC, SG, KMS, Secrets Manager, EC2) |
| 2. <code>es-test/vpn/</code> | -> Pritunl VPN server (baca VPC/SG dari #1) |
| 3. <code>es-test/monitoring/</code> | -> Kibana + dashboard (baca ES node IPs dari #1, SG VPN dari #2) |
| 4. <code>es-test/alerting/</code> | -> CloudWatch alarm + SNS + cron metric pusher (baca instance IDs dari #1) |

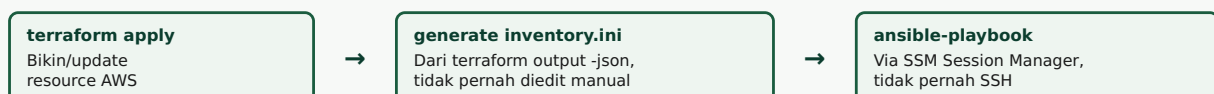
Setiap stack punya pola 3 langkah yang identik di `deploy.sh`-nya: 1. `terraform apply` — bikin/update resource AWS. 2. Generate `ansible/inventory.ini` otomatis dari `terraform output -json` (tidak pernah diedit manual). 3. `ansible-playbook` — install & konfigurasi software di instance, koneksi **selalu lewat SSM Session Manager** (`ansible_connection=community.aws.aws_ssm`), **tidak pernah SSH**.

Prinsip desain inti yang mengulang di semua stack: - **Zero SSH** — semua akses instance lewat AWS SSM, audit trail otomatis via CloudTrail, tidak ada port 22 terbuka di manapun. - **Tidak ada secret hardcoded** — semua password di-generate Terraform (`random_password`), disimpan di AWS Secrets Manager (encrypted-at-rest pakai KMS CMK), diambil instance saat runtime lewat IAM role yang scoped per-secret. - **Idempotent** — setiap playbook aman dijalankan ulang (guard pakai marker block, SAN check pada cert, `creates:` pada file). - **Fail-loud** — playbook didesain supaya gagal secara eksplisit kalau ada yang salah, bukan diam-diam "sukses" padahal rusak (lihat detail bug di Section 5).

DIAGRAM 1 — URUTAN PROVISIONING 4 STACK (DEPENDENCY VIA TERRAFORM_REMOTE_STATE)

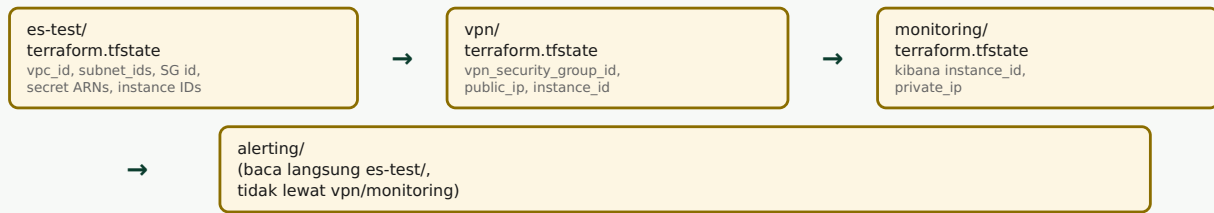


DIAGRAM 2 — POLA DEPLOY.SH (IDENTIK DI SETIAP STACK)



2. Penjelasan Source Code — Stack per Stack

DIAGRAM 3 — SETIAP STACK: APA YANG DIBACA DARI STACK LAIN (DATA FLOW ANTAR TERRAFORM STATE)



2.1 Stack `es-test/` (ElasticSearch 3-node, fondasi semua stack lain)

`terraform/main.tf` — membuat: - **VPC** `10.1.0.0/16` dengan 1 subnet publik (`10.1.0.0/24`) + 2 subnet privat (`10.1.1.0/24` , `10.1.2.0/24`) di 2 Availability Zone berbeda. - **NAT Gateway** di subnet publik — supaya node di subnet privat tetap bisa akses internet (yum repo, AWS API) tanpa IP publik sendiri. - **Security Group** (`es-test-sg`) dengan 3 rule terpisah (sengaja dipisah dari inline block — ada catatan bug soal ini, lihat Section 5): ingress 9200 dari `allowed_cidr`, egress all, dan ingress 9300 **self-referencing** (hanya member SG yang sama boleh saling bicara di port transport). - **KMS Customer-Managed Key** (`es_cmek`) dengan key rotation aktif — dipakai untuk enkripsi EBS **dan** Secrets Manager. - **3 secret di Secrets Manager**: `elastic-password`, `kibana-system-password`, `ca-bundle` (CA cert+key untuk TLS transport antar-node) — semua `recovery_window_in_days=0` supaya destroy/recreate cycle tidak kena block AWS. - **IAM role** (`es_role`) dengan izin scoped: `secretsmanager:GetSecretValue` hanya ke 2 ARN spesifik, `kms:Decrypt` hanya ke CMK itu, plus policy SSM managed core. - **S3 bucket** (`ansible_ssm_transfer`) — dipakai plugin koneksi `community.aws.aws_ssm` sebagai relay transfer file (bukan network path langsung, tetap zero-SSH). - **3x EC2 instance** (`for_each`, bukan `count`) tersebar di 2 subnet privat secara `% 2` — `user_data` minimal, cuma install `amazon-ssm-agent` (AMI ini tidak ship SSM agent default). Instalasi ElasticSearch sendiri **sengaja tidak** di `user_data` — dipindah ke Ansible karena `user_data` jalan sekali saat boot dan tidak tahu IP node lain (bug lama yang sudah diperbaiki, lihat Section 5).

`ansible/elasticsearch.yml` — playbook utama konfigurasi ES, task-task pentingnya: 1. Naikkan `vm.max_map_count` ke 262144 (wajib untuk ES bootstrap check). 2. Install ES 8.x dari repo resmi Elastic (`gpgcheck: true` — package signature diverifikasi). 3. Ambil CA bundle dari Secrets Manager, generate **sertifikat per-node** ditandatangani CA yang sama (`elasticsearch-certutil`) — dipakai untuk TLS HTTP (9200) **dan** transport (9300). 4. Simpan password keystore PKCS12 ke **ES keystore terenkripsi sendiri** (`elasticsearch-keystore add -x`), bukan di `elasticsearch.yml` plaintext. 5. Tulis file `unicast_hosts.txt` berisi **IP privat semua node** — ini mekanisme discovery (lihat Section 6). 6. Replace `elasticsearch.yml` dengan config minimal terkelola penuh via `blockinfile` (guard idempotent pakai marker comment). 7. Start/restart ES (hanya restart kalau config/cert berubah — supaya re-run tidak selalu restart). 8. Set password user `elastic` via `elasticsearch-reset-password`, lalu **verifikasi** password itu benar-benar bisa auth (bukan cuma percaya return code sukses — ini fix dari bug nyata, lihat Section 5). 9. Set juga password `kibana_system` (service account khusus Kibana — ES 8.x menolak user `elastic` untuk koneksi Kibana→ES).

`terraform/outputs.tf` — expose semua ARN/ID yang dibutuhkan stack turunan: `instance_ids`, `node_private_ips`, `elastic_secret_arn`, `kibana_system_secret_arn`, `ca_bundle_secret_arn`, `vpc_id`, `private_subnet_ids`, `es_security_group_id`, `es_cmek_arn`, `es_iam_role_name`, `ansible_transfer_bucket`.

`deploy.sh` — orkestrasi 3 langkah di atas. Menunggu SSM registration (polling `aws ssm describe-instance-information`, timeout ~5 menit) sebelum serah-terima ke Ansible, retry playbook run sampai 3x (mengatasi race condition SSM yang terobservasi saat testing).

2.2 Stack `es-test/vpn/` (Pritunl VPN)

`terraform/main.tf` — baca `terraform_remote_state` dari stack ES untuk dapat `vpc_id`, `public_subnet_id`, `es_security_group_id`. Bikin: - Security Group VPN: port 443 (admin UI) dari `admin_cidr`, port 1194/UDP (tunnel OpenVPN) dari `0.0.0.0/0` (autentikasi terjadi di layer VPN client-profile, bukan network ACL). - Rule tambahan di **SG ES** supaya VPN SG boleh akses 9200 (`source_security_group_id`, bukan CIDR publik). - 1x EC2 di subnet publik + Elastic IP.

`ansible/pritunl.yml` — install MongoDB + Pritunl dari repo resmi. **Catatan risiko yang didokumentasikan secara eksplisit**: `gpgcheck: false` untuk repo Pritunl karena GPG key upstream mereka 404 di semua path yang dicoba — trade-off yang disadari, bukan kelalaian (transport tetap HTTPS).

ansible/pritunl-provision.yml — **fully-automated app-level bootstrap**, tanpa UI manual: 1. Fix bug `mongodb_uri` kosong di `/etc/pritunl.conf` (default install Pritunl meninggalkan ini kosong meski service lapor "running"). 2. Reset password admin (`pritunl reset-password pritunl` — **username admin default adalah pritunl** , bukan `admin` , dikonfirmasi lewat query MongoDB langsung). 3. Enable REST API auth via update MongoDB langsung (tidak ada CLI subcommand untuk ini). 4. Jalankan `files/pritunl_provision.py` — script Python yang HMAC-sign request ke Pritunl REST API untuk bikin Organization + Server (UDP/1194) + User, lalu download profile `.ovpn` client via endpoint `/data/<org_id>/<user_id>/<server_id>.key`. 5. Simpan semua credential (admin password, API token/secret, org/server/user ID, `.ovpn` base64) ke **HashiCorp Vault lokal** (`kv/pritunl`) — tidak pernah dicetak ke stdout/log (`no_log: true` di semua task sensitif).

2.3 Stack `es-test/monitoring/` (Kibana + Dashboard)

terraform/main.tf — baca remote state ES **dan** VPN. Bikin: - Security Group Kibana: port 5601 hanya dari **SG VPN** (`source_security_group_id`) — Kibana **tidak pernah** reachable langsung dari internet, hanya lewat VPN. - Rule tambahan di SG ES supaya Kibana boleh akses 9200. - IAM role Kibana dengan izin baca 2 secret (`kibana_system` password + CA bundle) + `kms:Decrypt` . - 1x EC2 di subnet **privat** yang sama dengan node ES (tidak perlu IP publik sama sekali).

ansible/kibana.yml — install Kibana, trust CA ES (copy `ca_cert` ke `/etc/kibana/certs/`), konfigurasi `elasticsearch.hosts` ke ketiga node ES, auth pakai **kibana_system** (bukan `elastic` — ES 8.x menolak superuser untuk ini). Health check `GET /api/status` sampai benar-benar HTTP 200 (bukan toleransi 503 — lihat Section 5 soal bug ini).

ansible/kibana-dashboards.yml (ditambahkan belakangan, atas permintaan eksplisit untuk mengisi Kibana yang semula kosong) — provisioning **Saved Objects** (1 index pattern + 5 visualization + 1 dashboard) lewat Kibana Saved Objects Import API (`POST /api/saved_objects/_import?overwrite=true`), pakai **user elastic** (bukan `kibana_system` — service account itu hanya untuk koneksi Kibana→ES internal, tidak bisa dipakai memanggil API Kibana sendiri). File `ndjson`-nya statis (`files/kibana-saved-objects.ndjson` , checked-in ke repo, tanpa data sensitif) berisi definisi dashboard "ES Test Cluster Overview" (5 panel: cluster health, CPU/heap/disk per node, doc count, jumlah node). Set juga sebagai **default landing page** Kibana (`defaultRoute` setting) — jadi begitu login langsung tampil terisi.

2.4 Stack `es-test/alerting/` (CloudWatch + SNS + Cron Metric)

terraform/main.tf — baca remote state ES. Bikin: - SNS topic `es-test-alerts` + subscription email (**satu-satunya manual step di seluruh sistem** — AWS tidak punya API untuk auto-confirm subscription email). - **7 CloudWatch alarm**: `CPUUtilization>80%` (per node, 15 menit) dan `StatusCheckFailed>0` (per node, 2 menit) — native, tidak perlu agent. Plus **1 alarm custom** `ClusterHealthStatus>0` yang mengalarm ke metric custom (lihat di bawah). - Semua alarm `treat_missing_data = "breaching"` — kalau data hilang (instance mati/cron berhenti), itu **dianggap alarm**, bukan diam. - IAM policy tambahan (`cloudwatch:PutMetricData` scoped ke namespace `EsTest/Custom`) ditambahkan ke role ES yang sudah ada (bukan role baru).

ansible/es-metrics-cron.yml — install cron (setiap 1 menit) yang jalan di tiap node ES, melakukan **dua hal sekaligus**: 1. `curl _cluster/health` lokal → mapping status ke angka (`green=0`, `yellow=1`, `red=2`) → `aws cloudwatch put-metric-data` ke namespace `EsTest/Custom` — ini basis alarm cluster health. | true supaya kegagalan bagian ini tidak menggagalkan push metric CloudWatch yang jadi jalur kritis alerting. Juga membuat **index template** (`es-test-metrics-*`) dengan field mapping eksplisit **sebelum** cron pertama jalan — supaya tipe field benar untuk visualisasi Kibana (bukan auto-detect yang kadang salah tebak).

3. Cara Menggunakan & Integrasi

DIAGRAM 4 — URUTAN PROVISIONING OPERATOR (COMMAND-LEVEL)

1. `cd es-test/ && ./deploy.sh <allowed_cidr>`
ElasticSearch 3-node



2. `cd vpn/ && ./deploy.sh <admin_cidr>`
Pritunl VPN



3. `cd monitoring/ && ./deploy.sh`
Kibana + dashboard



4. `cd alerting/ && ./deploy.sh <alert_email>`
CloudWatch + SNS

3.1 Prasyarat

- AWS credentials dengan akses penuh ke VPC/EC2/IAM/KMS/SecretsManager/CloudWatch/SNS/S3/SSM pada akun target.
- `terraform`, `ansible`, AWS CLI, dan `session-manager-plugin` sudah ter-install dan ada di `PATH`.
- Tidak perlu SSH keypair sama sekali.

3.2 Urutan Provisioning (wajib berurutan)

```
1. es-test/           ./deploy.sh <allowed_cidr>      # ElasticSearch 3-node
2. es-test/vpn/       ./deploy.sh <admin_cidr>    # Pritunl VPN (baca state ES)
3. es-test/monitoring/ ./deploy.sh          # Kibana + dashboard (baca state ES+VPN)
4. es-test/alerting/   ./deploy.sh <alert_email>  # CloudWatch + SNS (baca state ES)
```

Urutan destroy adalah kebalikannya (alerting → monitoring → vpn → es-test), karena ketergantungan `terraform_remote_state`.

3.3 Integrasi Aplikasi ke ES (bukan hardcode password)

DIAGRAM 5 — ALUR APLIKASI MENGAMBIL PASSWORD ES (RUNTIME, BUKAN HARDCODE)

Aplikasi start di EC2/Lambda/ECS
di dalam VPC 10.1.0.0/16 (atau via Pritunl VPN)



pakai IAM role instance (bukan access key hardcode)

`sm.get_secret_value(SecretId=...)`
AWS Secrets Manager API call



IAM role harus di-grant `GetSecretValue` + `kms:Decrypt` scoped ke ARN ini

KMS decrypt (CMK es_cmek)
password dikembalikan plaintext ke proses aplikasi



password hanya hidup di memory proses

`Elasticsearch(basic_auth=("elastic", password))`
connect ke salah satu/ketiga node ES (9200, TLS)

Repo ini murni infra provisioning — tidak ada kode aplikasi contoh. Pola integrasi yang dimaksudkan untuk aplikasi klien mana pun:

1. **Penempatan jaringan:** aplikasi harus berjalan di dalam VPC yang sama (`10.1.0.0/16`), atau terhubung lewat Pritunl VPN. ES **tidak pernah** diexpose ke internet publik.
2. **Credential diambil runtime, bukan hardcode:** compute aplikasi (EC2/Lambda/ECS) perlu IAM role baru yang di-grant `secretsmanager:GetSecretValue` ke `elasticsearch/es-test/elastic-password` + `kms:Decrypt` ke CMK — scoped sama seperti pola role ES/Kibana yang sudah ada. Role ini **belum ada di repo** karena belum ada consumer app nyata.
3. **Password hanya hidup di memory proses aplikasi selama runtime** — tidak pernah ditulis ke file config, env var image, atau source control.

Contoh pola kode (Python):

```
import boto3
from elasticsearch import Elasticsearch

sm = boto3.client("secretsmanager", region_name="ap-southeast-3")
password = sm.get_secret_value(
    SecretId="elasticsearch/es-test/elastic-password"
)["SecretString"]

es = Elasticsearch(
    ["https://10.1.1.30:9200", "https://10.1.2.82:9200", "https://10.1.1.156:9200"],
    basic_auth=("elastic", password),
    verify_certs=False, # atau ca_certs=<path CA> untuk full verification
)
```

3.4 Akses Kibana (operator/reviewer)

```
http://<ip-privat-kibana>:5601
```

Hanya reachable setelah connect VPN Pritunl. Login `elastic` + password dari Secrets Manager. Begitu login, langsung diarahkan ke dashboard "ES Test Cluster Overview" — tidak perlu setup manual apa pun.

3.5 Verifikasi cluster sehat

```
SECRET=$(aws secretsmanager get-secret-value --secret-id elasticsearch/es-test/elastic-password --query SecretString --output text)
curl -k -u elastic:$SECRET https://<ip-node>:9200/_cluster/health?pretty
# expect: "number_of_nodes": 3, "status": "green"
```

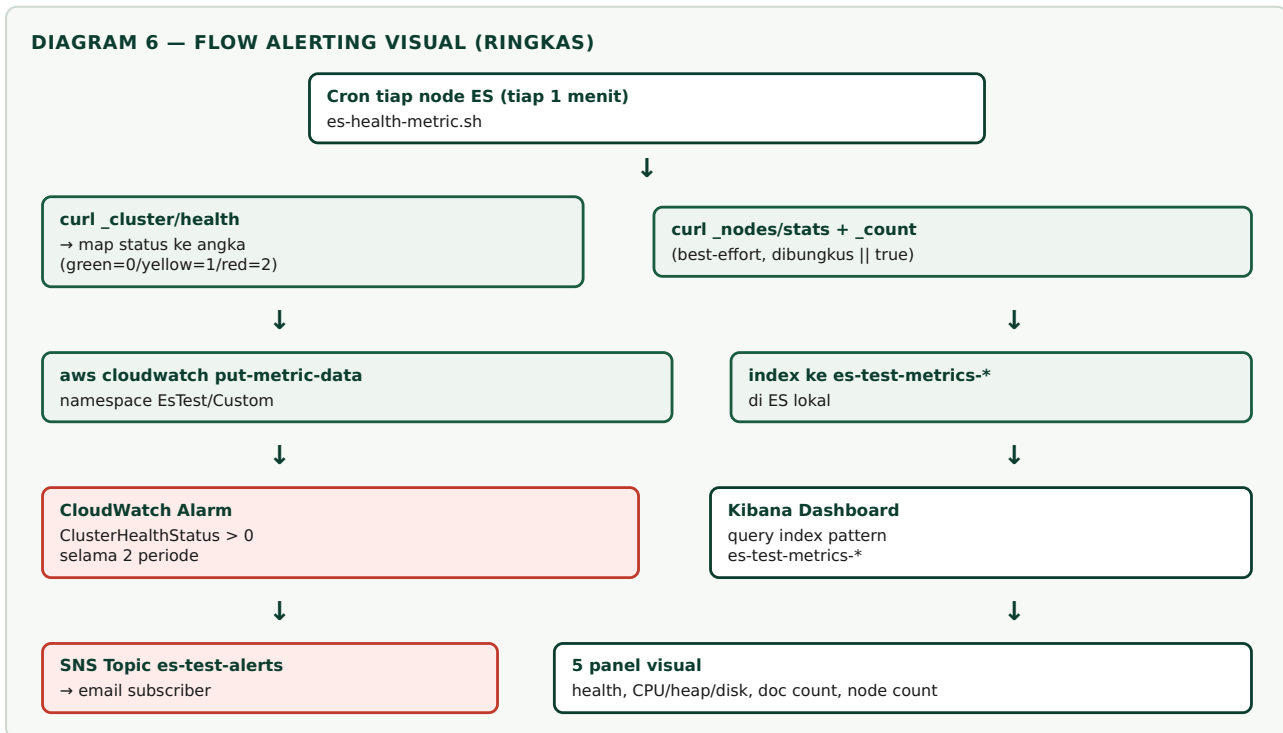
4. Monitoring & Alerting — Implementasi dan Flow

4.1 Lapisan Monitoring

Layer	Mekanisme	Deteksi apa
Host/instance (native)	CloudWatch <code>CPUUtilization</code> , <code>StatusCheckFailed</code>	Masalah di level EC2/hardware/network — tidak butuh agent apa pun
Cluster internal ES	Cron tiap node → <code>_cluster/health</code> → custom metric CloudWatch (<code>EsTest/Custom</code>)	CloudWatch sendiri tidak punya visibilitas ke status green/yellow/red ES — makanya perlu jembatan custom ini
Visual/ dashboard	Kibana (data dari <code>es-test-metrics-*</code> , diisi cron yang sama)	CPU/heap/disk per node, jumlah dokumen, jumlah node — divisualisasikan, bukan cuma alarm biner

4.2 Flow Alerting End-to-End

DIAGRAM 6 — FLOW ALERTING VISUAL (RINGKAS)



Detail flow lengkap (format ASCII, termasuk jalur native CloudWatch paralel):

```
[Tiap node ES, tiap 1 menit]
es-health-metric.sh
├── curl https://localhost:9200/_cluster/health (auth: elastic)
│   ├── mapping status → angka (green=0/yellow=1/red=2)
│   └── aws cloudwatch put-metric-data (namespace EsTest/Custom, metric ClusterHealthStatus)
│       └── [CloudWatch Alarm "es-test-cluster-health-not-green"] evaluates >0 selama 2
│           └── ALARM state → SNS topic "es-test-alerts" → email subscriber
├── periode
└── (best-effort, dibungkus || true) curl _nodes/stats + _count
    ├── index dokumen metrik ke es-test-metrics-<tanggal> di ES lokal
    ├── [Kibana dashboard "ES Test Cluster Overview"] query index pattern es-test-metrics-*
    └── render 5 panel visual (health, CPU/heap/disk, doc count, node count)

[Paralel, native CloudWatch tanpa cron]
AWS pantau tiap instance EC2 langsung
├── CPUUtilization > 80% selama 15 menit → Alarm → SNS → email
└── StatusCheckFailed > 0 selama 2 menit → Alarm → SNS → email
```

Kenapa dua jalur alerting terpisah (native EC2 + custom cron)? CloudWatch secara native cuma tahu kondisi instance (CPU, hidup/mati) — ia buta total terhadap kondisi internal ES (apakah shard ter-alokasi, apakah cluster quorum terbentuk, dst). Cron kecil ini adalah satu-satunya jembatan yang memberi CloudWatch visibilitas ke level aplikasi.

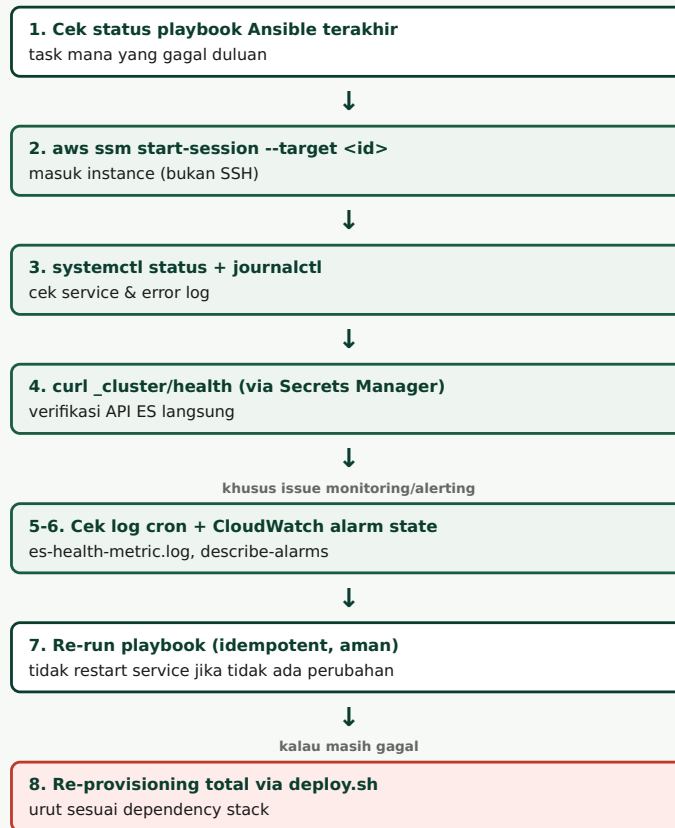
Kenapa `treat_missing_data="breaching"` di semua alarm? Supaya ketiadaan data (misal cron berhenti jalan, atau instance benar-benar unreachable sehingga tidak ada metric masuk sama sekali) **dianggap kondisi alarm**, bukan dianggap "tidak ada masalah karena tidak ada data". Default AWS adalah `notBreaching` — itu berbahaya untuk kasus failure total (silent death).

4.3 Kenapa dashboard-nya seperti itu (bukan Metricbeat)

Pilihan desainnya adalah cron+curl manual (bukan Metricbeat/agent resmi Elastic), karena cukup untuk membuktikan pipeline alerting end-to-end dalam scope take-home test, auditable (1 file bash yang bisa dibaca penuh), dan tidak menambah instalasi service baru di setiap node. Trade-off: metrik yang dikumpulkan dangkal (tidak ada GC pause time, rejected thread pool, shard-level detail) — kalau mau produksi nyata, ini yang pertama diganti dengan Metricbeat/CloudWatch Agent ES integration.

5. Cara Debugging Jika Ada Issue

DIAGRAM 7 — ALUR DIAGNOSIS UMUM (LANGKAH 1-8, LIHAT SECTION 5.2)



5.1 Daftar bug nyata yang pernah ditemukan + cara diagnosisnya (preseden untuk issue serupa)

Gejala	Root cause	Cara ketemu	Fix
Node ES tidak pernah membentuk cluster (<code>number_of_nodes: 1</code> terus)	<code>unicast_hosts.txt</code> tidak pernah terisi IP peer — <code>user_data</code> Terraform hanya tahu IP diri sendiri saat boot	Cek isi <code>/etc/elasticsearch/unicast_hosts.txt</code> di tiap node — kalau cuma 1 baris (IP sendiri), ini penyebabnya	Pindahkan populate file ini ke Ansible (<code>elasticsearch.yml</code> task "Populate discovery-file") — Ansible tahu semua node dari inventory di awal, bukan cuma 1
ES gagal start sama sekali, error bootstrap check	<code>vm.max_map_count</code> default (65530) terlalu rendah begitu <code>xpack.security</code> aktif	Lihat <code>journalctl -u elasticsearch</code> — error eksplisit soal <code>max_map_count</code>	<code>sysctl vm.max_map_count=262144</code> sebelum install
<code>elasticsearch-reset-password</code> lapor sukses (<code>rc=0</code>) tapi password yang disimpan di Secrets Manager tidak bisa dipakai login	Shell-injection: password random mengandung karakter <code>\$</code> diikuti huruf/angka (mis. <code>...\$a4...</code>) — saat displatkan langsung ke string shell via Jinja, bash menginterpretasikannya sebagai variable expansion dan diam-diam memotong/ mengubah nilainya	Tidak kelihatan dari log (<code>rc=0</code> , "successfully reset") — baru ketahuan saat verifikasi eksplisit auth check gagal	Lewatkan password via <code>environment:</code> Ansible (<code>ELASTIC_PW</code>), referensi sebagai <code>"\$ELASTIC_PW"</code> dalam tanda kutip tunggal di <code>printf</code> — bukan menyuntik literal ke string shell

Gejala	Root cause	Cara ketemu	Fix
Kibana stuck di HTTP 503 selamanya, playbook tetap melaporkan "success"	Kasus di atas, versi kibana_system — password yang ter-apply ke ES tidak cocok dengan yang disimpan di Secrets Manager (karena bug shell-injection yang sama), tapi task health-check lama menerima status 503 sebagai "pass" (ditoleransi sebagai "masih starting")	Manual curl <code>http://localhost:5601/api/status</code> lama setelah "deploy selesai" — baru ketahuan Kibana sebenarnya down permanen	(a) fix shell-injection yang sama seperti di atas, (b) ganti health-check jadi hanya terima status_code 200 — retry sampai itu tercapai, gagal loud kalau tidak pernah tercapai
Rule security group transport (9300, self-referencing) hilang setiap kali <code>allowed_cidr</code> di-apply ulang	Mixing inline <code>ingress / egress</code> block di <code>aws_security_group</code> dengan <code>aws_security_group_rule</code> terpisah untuk SG yang sama — Terraform block inline menimpa semua rule di grup itu setiap apply yang menyentuhnya	Diff terraform plan menunjukkan rule <code>transport_self</code> di-destroy padahal tidak diminta	Hapus semua inline ingress/egress dari resource <code>aws_security_group</code> , pindahkan semuanya jadi <code>aws_security_group_rule</code> terpisah
Operasi destroy→apply ulang Secrets Manager gagal (<code>InvalidRequestException: already scheduled for deletion</code>)	Default AWS recovery window 30 hari memblokir pembuatan ulang secret dengan nama sama	Error eksplisit dari AWS API	<code>recovery_window_in_days = 0</code> di semua secret (trade-off khusus demo/test, bukan untuk secret produksi nyata)
Pritunl service "running" tapi semua operasi admin/API gagal	<code>mongodb_uri</code> kosong di <code>/etc/pritunl.conf</code> meski mongod & pritunl sama-sama lapor status aktif	<code>pritunl get-mongodb</code> return kosong	<code>pritunl set-mongodb mongodb://localhost:27017/pritunl</code> lalu restart service
Login admin Pritunl pakai username <code>admin</code> selalu gagal	Username default superuser Pritunl ternyata pritunl , bukan <code>admin</code>	Query langsung <code>db.administrators.find()</code> di MongoDB	Pakai pritunl sebagai username di semua task yang reset password/API

5.2 Langkah debugging umum (untuk issue baru yang belum terdaftar di atas)

- Cek status playbook Ansible paling akhir** — semua task sensitif pakai `no_log: true`, jadi output sensitif tidak bocor ke log, tapi task non-sensitif (health check, debug message) tetap tercatat — baca urutan task yang gagal duluan.
- Masuk ke instance via SSM** (bukan SSH): `bash aws ssm start-session --target <instance-id> --region ap-southeast-3`
- Cek service status & log:**
`bash sudo systemctl status elasticsearch # atau: kibana, pritunl sudo journalctl -u elasticsearch -n 100 --no-pager`
- Verifikasi cluster health langsung dari node** (hindari print password — ambil dari Secrets Manager via AWS CLI, bukan hardcoded): `bash SECRET=$(aws secretsmanager get-secret-value --secret-id elasticsearch/es-test/elastic-password --region ap-southeast-3 --query SecretString --output text) curl -k -u elastic:$SECRET https://localhost:9200/_cluster/health?pretty`
- Verifikasi cron metric jalan** (untuk issue monitoring/alerting): `bash cat /var/log/es-health-metric.log # log setiap run cron sudo crontab -l # pastikan entry cron ada`
- Cek CloudWatch alarm state langsung:** `bash aws cloudwatch describe-alarms --region ap-southeast-3 --alarm-name-prefix es-test`
- Re-run playbook yang relevan** — semua playbook di repo ini idempotent, aman dijalankan ulang tanpa efek samping (tidak akan restart service/regenerate cert kalau tidak ada perubahan nyata).
- Kalau butuh re-provisioning total:** jalankan ulang `deploy.sh` stack terkait dari root,urut sesuai dependency (lihat Section 3.2).

6. Mekanisme Failover

Failover di sistem ini berlapis **3 layer independen**:

DIAGRAM 8 — 3 LAYER FAILOVER (INDEPENDEN, SALING MELENGKAPI)

Layer 1: Cluster ES (native)

Quorum 2/3, replikasi shard, spread 2 AZ, mTLS transport

Layer 2: Infra (CloudWatch)

StatusCheckFailed, CPUUtilization>80%

Layer 3: Aplikasi (custom cron)

ClusterHealthStatus, treat_missing_data=breaching

6.1 Layer cluster ES (native)

- Semua 3 node **master-eligible + data** (`node.roles: [master, data]`) — tidak ada differensiasi read/write per node, semua simetris.
- `cluster.initial_master_nodes` mendaftarkan ketiga node sejak awal — quorum majority **2 dari 3** node diperlukan untuk memilih master baru kalau 1 node mati.
- Replikasi shard antar-node otomatis (ES native) — data tidak hilang kalau 1 node turun, selama shard primary+replica tidak keduanya ada di node yang sama node yang mati.
- Node tersebar di **2 Availability Zone berbeda** (% 2 distribusi) — AZ-level fault tolerance, bukan cuma node-level.
- Transport layer (9300) pakai **mutual TLS** antar node (CA sama) — diperlukan khusus untuk cluster multi-node (single-node tidak butuh ini).

6.2 Layer infra (CloudWatch native)

- `StatusCheckFailed` alarm (2 menit) — deteksi kegagalan hardware/network-level EC2 yang dideteksi AWS sendiri, independen dari apakah ES masih merespons atau tidak.
- `CPUUtilization>80%` (15 menit) — sinyal awal node kelebihan beban sebelum benar-benar mati.

6.3 Layer aplikasi (custom cron → CloudWatch → SNS)

- Cron tiap node poll `_cluster/health` sendiri — kalau status bukan `green`, alarm `ClusterHealthStatus` menyala, memberi sinyal masalah di level cluster logic (unassigned shard, split brain, dll) yang tidak terlihat dari layer infra saja.
- `treat_missing_data="breaching"` — node yang benar-benar mati (cron berhenti total) **juga** memicu alarm, bukan hilang diam-diam.

6.4 Prosedur Replace Node Zero/Minimal-Downtime (manual, terdokumentasi, bukan otomatis)

DIAGRAM 9 — URUTAN REPLACE-NODE MINIMAL-DOWNTIME

1. Freeze shard allocation

`cluster.routing.allocation.enable=none`



2. Stop/terminate/replace instance

Terraform: `taint + apply`



3. Node baru rejoin otomatis

Ansible re-provisioning (`cert + unicast_hosts.txt` baru)



4. Re-enable allocation

`cluster.routing.allocation.enable=all`



5. Tunggu status green sebelum node berikutnya

rolling, satu per satu, TIDAK paralel

```
# 1. Freeze shard allocation sebelum menyentuh node (hindari reshuffle shard sia-sia)
curl -k -u elastic:$SECRET -X PUT https://<node-ip>:9200/_cluster/settings \
  -d '{"transient":{"cluster.routing.allocation.enable":"none"}}'

# 2. Stop/terminate/replace instance target (Terraform: taint + apply)

# 3. Node baru rejoin otomatis (Ansible re-provisioning via deploy.sh menghasilkan
#   cert baru + unicast_hosts.txt baru secara idempotent)

# 4. Re-enable allocation
curl -k -u elastic:$SECRET -X PUT https://<node-ip>:9200/_cluster/settings \
  -d '{"transient":{"cluster.routing.allocation.enable":"all"}}'

# 5. Tunggu status kembali green sebelum lanjut ke node berikutnya (rolling, satu per satu, TIDAK paralel)
```

Karena quorum tetap terjaga (2 dari 3 masih hidup) selama prosedur ini, cluster tetap **fully available** (green→yellow→green per node), bukan downtime penuh.

Yang belum diotomasi (sengaja, batas waktu take-home): prosedur di atas belum di-wire jadi satu command Terraform/Ansible tunggal — produksi nyata idealnya pakai Auto Scaling Group dengan `instance_refresh` policy.

Dokumen ini adalah ringkasan teknis lengkap repo `es-test/`. Untuk jawaban soal take-home test secara verbatim (deskripsi solusi, resource, waktu, feedback), lihat `docs/02_SUPPORTING_INSTRUCTIONS_EN.md` di repo.